Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Görüntü sıkıştırmaya genel bakış
  - Görüntüde de kayıplı ve kayıpsız sıkıştırma vardır. Slayta göre kayıpsız sıkıştırma genellikle pikseller arası korelasyonun düşük olduğu şekil gibi görüntülerde, kayıplı sıkıştırma fotoğraf gibi karmaşık ve yüksek korelasyonlu görüntülerde kullanılır (ekrandan, 00:07:00).
  - Kayıpsız görüntü sıkıştırma olasılık tabanlı (Huffman, aritmetik kodlama), sözlük tabanlı (LZ77, LZW) ya da ikisini birlikte kullanan hibrit bir yöntemle yapılabilir. PNG, LZ77 ile Huffman'ı birleştiren DEFLATE yöntemini kullanır. JPEG-LS ve JBIG özellikle kayıpsız görüntü sıkıştırma için tasarlanmıştır (ekrandan, 00:07:00).
  - Görüntüde kullanılan yöntemler metinde görülen yöntemlerin aynısıdır; fark, verinin piksel ve renk olması ve kayıplı sıkıştırmada bazı ön işlem adımlarının bulunmasıdır.
- RLE (Run Length Encoding)
  - Bir değer ardışık olarak çok kez tekrar ediyorsa değer bir kez, ardından tekrar sayısı kodlanır. Metinde başarısızdır, çünkü metinde aynı karakter nadiren art arda tekrar eder; görüntüde ise iyidir. BMP, PCX ve TIFF RLE kullanır (ekrandan, 00:11:45).
  - 1 bitlik (siyah-beyaz) bir satırda 640 piksel: 200 beyaz, 240 siyah, 200 beyaz ise satır (200, 240, 200) diye kodlanır. Çözücü ilk pikselin rengini bilmelidir: ya kodlamanın başında belirtilir ya da ilk piksel her zaman beyaz kabul edilir (ekrandan, 00:11:45).
  - Kayıplı RLE: 2-3 piksellik kısa koşular gözün ayırt edemeyeceği renklerse komşu uzun koşunun rengine çevrilir.
- Faks standartları
  - CCITT T4 (Group 3) RLE ve Huffman kullanır. İlk piksel her zaman beyazdır. 64'ten büyük tekrar sayıları 64'e bölünür; bölüm ve kalan farklı Huffman tablolarıyla kodlanır. Bu kodlama Modified Huffman (MH) ya da G31D olarak bilinir (ekrandan, 00:20:00).
  - IBM'in 1979'da geliştirdiği MMR (Modified Modified READ), G32D olarak bilinir; bir önceki satırı referans alarak daha yüksek sıkıştırma sağlar, ama performansı sonradan geliştirilen JBIG'in gerisinde kalmıştır (ekrandan, 00:20:00).
- ZIP, GZIP ve PK programları
  - ZIP/GZIP: Giriş verisi kodlayıcının seçtiği, keyfî uzunlukta bloklara ayrılır; her bloğun Huffman ağacı ayrıdır ve bloğun başında saklanır. LZ77'nin bir türevi (32 KB kayan sözlük, 258 byte kaynak kod alanı) ile statik Huffman birlikte kullanılır (ekrandan, 00:30:00).
  - PKZip (Phil Katz), UnZip, Arc, Lite: PKArc'ta LZW ve Huffman, ayrıca dinamik LZW ile RLE ve LZ77 ile Shannon-Fano birleşimleri vardır (ekrandan, 00:30:00).
- JPEG-LS (kayıpsız JPEG)
  - ISO/IEC tarafından geliştirilmiştir (1998); kayıpsız ya da az kayıplı sıkıştırır. Standardın temeli Hewlett-Packard'ın LOCO-I (LOw COmplexity LOssless COmpression for Images) algoritmasıdır (ekrandan, 00:31:40).
  - Renkli görüntüde R, G, B paletlerinin her birine gri tonlu görüntüdeki işlem ayrı ayrı uygulanır. Kodlayıcı sol üst pikselden sağ alt piksele satır tarama düzeninde ilerler ve sekiz komşudan yalnız dördünü kullanır: W, NW, N, NE (ekrandan, 00:31:40).
  - Birinci aşamada G tahmini yapılır, ikinci geçişte (NW, W), (NW, N), (N, NE) piksel çiftlerinin farklarına göre düzeltme yapılır (ekrandan, 00:40:00).
- JPEG

  - DCT tabanlı bir standarttır; kayıp oranı Q (kalite) faktörüyle ayarlanır. 24 bit renk derinliğindeki fotoğraflarda çok iyi sonuç verir. Birçok yöntemin birleşimi olduğu için sıkıştırma ve açma süresi tek algoritmalı kayıpsız yöntemlerden uzundur (ekrandan, 00:43:30).
  - Mimari: RGB→YUV (isteğe bağlı), örneklemeyi azaltma, 8×8 bloklara bölme, DCT, nicemleme, zigzag tarama, DC'ye DPCM, AC'ye RLE, ardından QM kodlaması; JPEG dosyasında başlık, tablolar ve veri bulunur (ekrandan, 00:50:00).
- JPEG 2000, GIF, TIFF ve PNG
  - JPEG 2000 DCT yerine wavelet dönüşümü kullanır; kayıplı ve kayıpsız sıkıştırma yapabilir (ekrandan, 01:12:30).
  - GIF: CompuServe (1987), LZW tabanlı, en fazla 256 renk; 87a ve 89a sürümleri vardır (ekrandan, 01:14:00).
  - TIFF: Aldus (telif hakkı Adobe'de), 5.0 sürümünde LZW, 6.0 sürümünde JPEG; 24 bit renk desteği (ekrandan, 01:16:30).
  - PNG: Patent sorunu olmayan açık format (1996), 48 bit gerçek renk, 16 bit gri ton; altyapısında ZLIB ve DEFLATE vardır (ekrandan, 01:18:00).

Hocanın Özellikle Vurguladığı Kısımlar

- Görüntü sıkıştırmanın altında metinde görülen yöntemler yatar
  - Bu yüzden konu kısa sürer; Huffman, Shannon-Fano, LZ77, LZW ve RLE yeniden anlatılmaz, yalnız görüntüde nasıl kullanıldıkları gösterilir.
- Siyah-beyaz bir resimde JPEG kullanmak mantıksızdır
  - Hocaya göre siyah-beyaz resimde RLE'den daha iyi sonuç veren bir yöntem yoktur. JPEG 24 bit renkli fotoğraflar içindir.
- Faks kayıpsız sıkıştırılmak zorundadır
  - Faksa gönderilen metindir; kayıp olursa örneğin bir "T" harfinin çizgisi kaybolup "L" gibi okunabilir. Bunu düzeltmek doğal dil işleme gerektirir.
- Küçük bloklara bölmek sıkıştırmayı iyileştirir
  - JPEG bütün resme değil, 8×8 (ya da daha büyük) blokların her birine ayrı ayrı zigzag tarama uygular. Hoca, bütün resme tek zigzag uygulamak yerine küçük bloklarla çalışmanın sıkıştırma performansını artıracağını vurguladı.
- YUV'a geçişin nedeni insan gözüdür
  - Göz parlaklığı (Y) iyi ayırt eder, renklilikteki (U, V) küçük farkları ayırt edemez. Bu yüzden U ve V azaltılabilir, Y korunur.
- Kayıp nicemlemede oluşur
  - DCT katsayıları Q matrisine bölünüp yuvarlanır; yuvarlama yüzünden orijinale tam dönülemez. Q matrisleri hazırdır, kalite oranına göre seçilir.
- Hazır algoritmaları kullanmak yeterli olabilir, ama özel veride iç yapıyı bilmek gerekir
  - Transformlar ve nicemleme yöntemleri başka derslerin konusudur. Veri çok özelse kodda nereyi değiştireceğini bilmek için algoritmaların iç yapısı hakkında bilgi gerekir.

Kısa Tekrar Notları

- Kayıpsız: düşük korelasyonlu şekil görüntüleri; kayıplı: fotoğraflar.
- PNG = DEFLATE = LZ77 + Huffman.
- RLE: (200, 240, 200); ilk piksel beyaz kabul edilir, değilse başa 0 yazılır ya da başlıkta belirtilir.
- MH (G31D): 0-63 arası tekrar sayıları için bir tablo, 64'ün katları için ayrı tablo. MMR (G32D): önceki satırı referans alır.
- ZIP/GZIP: blok başına ayrı Huffman ağacı; 32 KB sözlük, 258 byte kaynak kod alanı.
- JPEG-LS: W, NW, N, NE komşuları; iki aşamalı tahmin; $9^3 = 729$ bağlam.
- JPEG: YUV → down-sampling → 8×8 → −128 → DCT → $C = \text{round}(D/Q)$ → zigzag → RLE/DPCM → entropi kodlaması.
- $Y = 0{,}299R + 0{,}587G + 0{,}114B$.
- Down-sampling: 2×2 blok için 4:4:4 yerine 4:2:2 ya da 4:1:1.
- Ters işlem: $R = Q \cdot C$, $N = \text{round}(M^T R M) + 128$.

- JPEG 2000: wavelet; GIF: LZW, 256 renk; TIFF: LZW/JPEG/ZIP, 24 bit; PNG: açık, ZLIB/DEFLATE.

Detaylı Açıklamalar

- **RLE ve görüntü.** RLE'nin metinde başarısız olmasının nedeni, metinde aynı karakterin art arda ikiden fazla tekrar etmesinin nadir olmasıdır; ancak boşluklar ya da uzun sıfır dizileri tekrar edebilir. Görüntüde ise aynı renkteki pikseller sık sık art arda gelir. Hocaya göre BMP, PCX ve TIFF gibi formatların desteklediği renk sayısı JPEG'e göre daha kısıtlı olduğundan aynı rengin art arda gelme sıklığı yüksektir ve RLE iyi sonuç verir. Siyah-beyaz bir satırda kodlayıcı yalnız tekrar sayılarını yazar: ilk sayı beyaz, ikincisi siyah, üçüncüsü beyaz koşunun uzunluğudur. Satır siyahla başlıyorsa başa 0 yazılır, yani "0 beyaz" denir. Gri tonlu bir resimde başlangıç rengi başlıkta (header) belirli bir byte'ta saklanır; çözücü bu bilgiyi oradan okur.
  - Ders sonrası not: Hoca BMP ve TIFF'in çok renk desteklemediğini söyledi (00:12:38). Slayttaki TIFF bölümüne göre TIFF 24 bit renk derinliğini destekler (ekrandan, 01:16:30).
- **Modified Huffman.** Faks makinesi metni bir görüntü gibi görür; yazının siyah kısımları siyah piksellere, boşluklar beyaz piksellere karşılık gelir. Faksta piksel yerine pel denir. Tekrar sayılarının olasılıkları çok çeşitli olduğu için iki Huffman tablosu kullanılır: biri 0-63 arası tekrar sayıları, diğeri 64'ün katları için. Örneğin 65 uzunluğunda bir koşu $65 = 64 \cdot 1 + 1$ olarak, 64 için ikinci tablodan, 1 için birinci tablodan kod alınarak yazılır.
  - Ders sonrası not: Hoca örneği "55 tane ... 54'e göre mod alıyordum, 1 kalıyordu" diye söyledi (00:18:02). Anlatılan tablo yapısına göre örnek büyük olasılıkla 65 ve 64'tür ($65 \bmod 64 = 1$).
- **MMR.** İki boyutlu kodlamada üstteki satır referans, alttaki satır kodlanan satırdır. Referans ve kodlanan satırdaki geçiş noktalarının (a1, b1 gibi) konumlarına göre yatay ve dikey kipler kullanılır; hoca üçüncü kipin adını hatırlamadı.
- **JPEG-LS tahmini.** Slayttaki birinci aşama (ekrandan, 00:40:00):

$$G = \begin{cases} \min(W, N) & \max(N, W) \leq NW \\ \max(W, N) & \min(N, W) < NW \\ W + N - NW & \text{diğer durumlarda} \end{cases}$$

- Slayttaki örnek: $N > W$ ve $N \leq NW$ ise bu yatay bir kenarı gösterir ve tahmin olarak W kullanılır. İkinci geçişte her piksel çiftinin farkı 9 gruptan birine sınıflandırılır; üç çift için $9^3 = 729$ farklı durum çıkar. Her durum kendi düzeltme değerini saklar; algoritma sonuç tahminiyle gerçek değer arasındaki farkı kodlar.
- Ders sonrası not: Slayttaki ikinci satırın koşulu büyük olasılıkla $NW \leq \min(N, W)$ olmalıdır (LOCO-I'nin medyan kenar tahmincisi); slaytta "$\min(N, W) < NW$" yazıyordu ve bu, ilk satırla çakışır.
- Ders sonrası not: Hoca 729'u "ortadaki pikselle 9 durum, 3 çift, 9^3" diye açıkladı (00:41:31). Slayta göre 9, piksel sayısı değil, her çiftin farkının sınıflandırıldığı grup sayısıdır.
- **Renk dönüşümü.** Slayttaki dönüşüm (ekrandan, 00:52:30): $Y = 0{,}299R + 0{,}587G + 0{,}114B$, $U = -0{,}169R - 0{,}331G + 0{,}500B$, $V = 0{,}500R - 0{,}149G - 0{,}081B$. YUV, R, G, B arasındaki korelasyonu azaltır ve rengi parlaklık (luminance) ile renklilik (chrominance) olarak ayırır.
  - Ders sonrası not: Slayttaki V satırındaki $-0{,}149G$ büyük olasılıkla $-0{,}419G$ olmalıdır. Gri bir piksel ($R = G = B$) için renklilik sıfır olmalıdır; $0{,}500 - 0{,}419 - 0{,}081 = 0$ iken slayttaki katsayıların toplamı $0{,}27$'dir (python ile kontrol edildi).
- **Down-sampling.** 2×2 boyutlu 4 piksellik bir blokta Y, U ve V için 4'er değer (4:4:4) yerine 4:2:2 ya da 4:1:1 kullanılabilir (ekrandan, 00:52:30). Y dört piksel için ayrı ayrı korunur, U ve V ikişer ya da birer değere indirilir.
  - Ders sonrası not: Hoca 4:4:4'teki sayıları "4 bit" diye anlattı (00:57:16). Bu sayılar bit değil, blok başına alınan örnek (değer) sayısıdır.
- **DCT ve nicemleme.** Blok değerleri 0-255 arasındadır; DCT $[-128, 127]$ aralığında çalıştığı için her değerden 128 çıkarılır ($0 - 128 = -128$, $255 - 128 = 127$) ve T matrisi elde edilir. $M_{0,j} = 1/\sqrt{N}$, $i > 0$ için $M_{i,j} = \sqrt{2/N}\cos[(2j+1)i\pi/(2N)]$; N blok boyudur (8

ya da 32). D matrisi nicemlenir: $C_{i,j} = \text{round}(D_{i,j}/Q_{i,j})$ (ekrandan, 01:07:40). Kayıp bu yuvarlamada oluşur. C'nin sağ alt köşesi sıfırla dolar; zigzag taramadan sonra değişken uzunluklu kodlama uygulanır. Geri dönüşte $R_{ij} = Q_{ij} \cdot C_{ij}$ ve $N = \text{round}(M^T RM) + 128$ (ekrandan, 01:01:00).
  - Ders sonrası not: Slaytta ileri dönüşüm $D = M^T TM$ yazıyordu (ekrandan, 01:00:00). Slayttaki M tanımıyla python'da denendiğinde doğru iki boyutlu DCT'yi $D = MTM^T$ verdi; ters dönüşüm için slayttaki $M^T RM$ doğrudur.
  - Ders sonrası not: Hoca bir yerde gözün alçak frekansları göremediğini ve yüksek frekansları kullanacağını söyledi (00:49:43), bir yerde de "DC'lerin birçoğu sıfır olarak kaybolacak" dedi (00:50:56). Slayta göre göz yüksek frekansları algılayamaz ve bunlar atılır; sıfırlanan katsayılar yüksek frekanslı AC katsayılarıdır, DC katsayısı korunur.
  - Ders sonrası not: Hoca zigzag şeklinde DC konumundaki katsayının "kesinlikle 0" olduğunu söyledi (01:01:32). Şekildeki 0, DC'nin zigzag sıra numarasıdır; değeri değil.
- **Kodlama adımı.** Hocaya göre sıfırlar art arda geldiği için AC katsayılarına RLE, DC katsayılarına Huffman uygulanır. Slayttaki mimaride DC için DPCM, AC için RLE ve ardından QM kodlaması yer alır (ekrandan, 00:50:00).
- **JPEG 2000.** Resmi dört alt banda böler (LL, LH, HL, HH filtreleri) ve bunları yeniden dörde bölerek seviyelere göre kayıp uygular. Hocaya göre DCT'den daha iyi sıkıştırır ve lisanslıdır.
  - Ders sonrası not: Hoca wavelet tabanlı JPEG 2000'in DCT'den daha hızlı çalıştığını söyledi (01:12:52). JPEG 2000 büyük olasılıkla JPEG'den daha fazla hesaplama gerektirir; üstünlüğü hız değil, düşük bit oranlarındaki kalitedir.
- **Formatlar.** GIF 256 renkle sınırlıdır, LZW kullanır, katmanlı görüntü saklayabilir: önce yalnız alçak frekanslar, ardından yüksek frekanslar eklenerek resim netleşir; böylece büyük bir resmin aranan resim olup olmadığı tamamı açılmadan anlaşılabilir. TIFF, LZW'nin patent sorunları yüzünden ZIP sıkıştırmasını da eklemiştir; JPEG'e göre daha az küçültür ama hızıyla bu açığı kapatır. PNG, GIF ve LZW'nin patent sorunları yüzünden geliştirilmiş, ücretsiz bir kayıpsız formattır.
  - Ders sonrası not: Hoca TIFF'in yer daha önemliyse tercih edildiğini söyledi (01:16:34). Slayta göre TIFF, zaman yerden daha önemliyse hızı yüzünden tercih edilir.
  - Ders sonrası not: Hoca DEFLATE'in içinde "LZW ve Huffman" olduğunu söyledi (01:17:39). Slayta ve hocanın ders başındaki anlatımına göre DEFLATE, LZ77 ile Huffman'ın birleşimidir (ekrandan, 00:07:00).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.